

If you're trying to break into AI engineering in 2026 or level up from where you are, here are 5 portfolio projects that will genuinely make a difference when you're sitting in front of a hiring manager.

I'm not talking about tutorial projects where you deploy a basic chatbot. I'm talking about work that makes someone look at your GitHub and say, "this person actually understands how production AI systems work."

The 5 Projects:

Project 1: Production RAG Application

Build a domain-specific "Ask My Docs" system with hybrid retrieval (BM25 + vector search), cross-encoder reranking, citation enforcement, and a CI-gated evaluation pipeline. This is the most common pattern in enterprise AI right now.

- LangChain's Official RAG Tutorial - A step-by-step guide to building retrieval chains:

<https://docs.langchain.com/oss/python/langchain/rag>

- Cohere: Rerank Guide - Explains how cross-encoders work and why they improve retrieval precision: <https://www.google.com/search?q=https://docs.cohere.com/docs/rerank-guide>

- Ragas Documentation - Learn how to evaluate faithfulness, answer relevance, and context precision: <https://docs.ragas.io/>

- If you are a beginner in LangChain/ LangGraph check out my workshop which will give you deep-dive into it along with hands-on experience:

https://topmate.io/the_gen_academy/1855779

- Here is a deep-dive RAG workshop I put together, it is in 2 parts

- RAG Foundations: https://topmate.io/aishwarya_srinivasan/1662461

- Advanced RAG Workshop: https://topmate.io/aishwarya_srinivasan/1750331

Project 2: Local SLM App with Ollama

Run models entirely offline. Benchmark inference performance. Compare 3 models on the same hardware. Document the quality-vs-speed tradeoffs. Privacy, latency, and cost constraints are real- show you understand them.

- Ollama Official GitHub & Quickstart - The easiest way to get Mistral or Llama running on your machine: <https://github.com/ollama/ollama>

- FastAPI Official Tutorial - Essential for wrapping your local model into a usable API:

<https://fastapi.tiangolo.com/tutorial/>

- Instructor (Python Library) - While she mentioned Pydantic, the Instructor library is the industry standard for enforcing Pydantic schemas on LLM outputs and handling retries gracefully: <https://python.useinstructor.com/>

- Hugging Face: Quantization Guide - A great read on how GGUF, Q4, and Q5 formats shrink models without completely destroying quality:

https://huggingface.co/docs/optimum/concept_guides/quantization

Project 3: Monitoring & Observability

Add tracing, latency tracking (p50/p95), cost-per-request, and quality metrics to your RAG system. Build regression gating into CI. This is 70% of production AI work that nobody puts in their portfolio.

- Here is a AI Evals workshop where we cover all you need to know about Evals and a hands-on lab: https://topmate.io/the_gen_academy/1962949

- Langfuse Documentation - Highly recommended as it's open-source. Their docs on "Tracing" and "Scores/Evaluations" are exactly what you need for this project:

<https://langfuse.com/docs>

- LangSmith Walkthrough - LangChain's native observability tool; great for understanding prompt versioning and tracing: <https://docs.smith.langchain.com/>

- Eugene Yan's "Patterns for Building LLM Systems" - A must-read article on how production AI systems are architected, monitored, and evaluated:

<https://eugeneyan.com/writing/llm-patterns/>

Project 4: Fine-Tuning with LoRA & DPO

Fine-tune for a specific task (JSON extraction or tool-calling). Use LoRA/QLoRA for efficient training. Add preference tuning with DPO. Show before-and-after metrics with actual numbers.

Datasets you can use for FT:

- SFT data:

- - HuggingFaceH4/ultrachat_200k – High-quality multi-turn chat, filtered from ShareGPT-style logs; widely used in open-source instruction models and supported out of

the box by Oumi SFT tooling:

https://huggingface.co/datasets/HuggingFaceH4/ultrachat_200k

-- Nemotron-Instruction-Following-Chat-v1 - Large, high-quality, commercially-usable instruction-following and chat dataset from NVIDIA, combining verifier-filtered assistant conversations and structured-output examples for post-training LLMs

<https://huggingface.co/datasets/nvidia/Nemotron-Instruction-Following-Chat-v1>

-- GSM8K – A dataset of 8.5k high-quality, linguistically diverse grade-school math word problems designed to test multi-step reasoning in language models:

<https://huggingface.co/datasets/openai/gsm8k>

-- Good Preference dataset – argilla/distilabel-intel-orca-dpo-pairs is a curated preference dataset (prompt, chosen, rejected) derived from Intel/orca_dpo_pairs, cleaned and enriched for better DPO-based chat alignment of open-source LLMs:

<https://huggingface.co/datasets/argilla/distilabel-intel-orca-dpo-pairs>

Coding Dataset:

-- OpenCodeInstruct – 5M diverse Python coding instruction–response pairs (questions, solutions, tests, execution feedback, and quality judgments) for training general-purpose code LLMs: <https://huggingface.co/datasets/nvidia/OpenCodeInstruct>.

-- OpenCoder opc-sft-stage1 – A curated multi-source supervised fine-tuning mix focused on code understanding and generation, used to train the OpenCoder models:

<https://huggingface.co/datasets/OpenCoder-LLM/opc-sft-stage1>

Project 5: Real-Time Multimodal Application

Build a voice assistant or streaming pipeline. Decompose end-to-end latency into a detailed budget. Add graceful degradation and timeout handling. Show you understand real-time systems.

- Orchestration Framework:

-- Pipecat AI - An incredible open-source framework specifically built for creating real-time voice and multimodal AI agents. It handles the WebSockets and latency tracking:

<https://github.com/pipecat-ai/pipecat>

- Speech-to-Text (ASR) & Text-to-Speech (TTS):

-- Deepgram API Documentation - Excellent guides on streaming audio transcription:

<https://developers.deepgram.com/>

-- ElevenLabs Python API Guide - Documentation specifically on using WebSockets to stream generated voice back to the user instantly:

<https://elevenlabs.io/docs/api-reference/websockets>

These 5 projects tell a cohesive story: you build AI systems that are reliable, measurable, and production-ready.

Which project are you tackling first? Drop a comment below.